

Lecture 1: Introduction to Parallel and Distributed Computing

1. Computing History & Moore's Law

- **Early Era:** ENIAC/UNIVAC used vacuum tubes, taking up entire rooms. Mainframe era followed with IBM System/360 (1960s).
- **Personal Computers:** Apple II and IBM PC popularized computing.
- **Moore's Law:** Predicts transistor count on chips doubles every two years, driving processing power growth.

2. Types of Computing

- **Serial Computing:** A single CPU executes instructions one after another. Simple but slow.
- **Parallel Computing:** Multiple CPUs solve parts of one single problem at the same time.
 - **Advantages:** Faster computation, efficient use of resources.
 - **Disadvantages:** High programming complexity, synchronization issues, and higher hardware cost (more cores/GPUs).
 - **Examples:** Multi-core CPUs, GPUs, supercomputers, scientific simulation, AI/ML training.
- **Distributed Computing:** Multiple independent computers connected via a network acting as one system.
 - **Advantages:** High fault tolerance (one fails, others keep working), cost-effective scaling via commodity hardware.
 - **Disadvantages:** Network latency/overhead, complex management, security vulnerabilities.
 - **Examples:** Cloud systems (clusters), microservices, Hadoop/Spark-style clusters, IoT networks, blockchains.
- **Parallel vs Distributed:** Parallel = single computer, multiple processors, shared/distributed memory, bus communication. Distributed = multiple computers, distributed memory only, message passing over network.

3. Parallel Memory Architectures

- **Shared Memory:** All processors use one global memory. Easy to program, but poor scalability due to memory path contention.
- **Distributed Memory:** Each processor has local memory and communicates via messages. Highly scalable, but harder to program (explicit data movement).
- **Hybrid Memory:** Shared memory within a node, distributed communication across nodes. Offers best scalability and performance but maximum complexity.

4. Communication Concepts

- **Latency vs Bandwidth:** Latency is the startup delay to send a small message; bandwidth is the data transferred per unit time.
- **Synchronous vs Asynchronous:** Synchronous is blocking (task waits); asynchronous is non-blocking.
- **Point-to-Point vs Collective:** Point-to-point is between two tasks; collective involves all tasks in a group.

Lecture 2: Flynn's Taxonomy & Parallelism

1. Flynn's Taxonomy

- **Instruction Stream:** The sequence of instructions fetched/executed by a CPU (conceptually tied to a program counter).
- **Data Stream:** The sequence of data items those instructions operate on.
- **SISD:** Single Instruction, Single Data. Traditional uniprocessor (old PCs). Example: single-core sequential machine (classic Von Neumann style).
- **SIMD:** Single Instruction, Multiple Data. Exact same operation runs on different data simultaneously. Best for vectors, matrix math, GPUs.
 - **Array Processor:** Each parallel processing unit has its own separate memory and register file.
 - **Pipelined Processor:** Read data from central resource (typically register file in modern CPUs), process fragments, write results back.
 - **Associative Processor:** Each parallel processing unit independently decides whether to execute or skip (predicated/masked SIMD).
- **MISD:** Multiple Instruction, Single Data. Different operations on the exact same data stream. Rare, rarely implemented commercially. Example: $Z = \sin(x) + \cos(x) + \tan(x)$.

- **MIMD:** Multiple Instruction, Multiple Data. Entirely different instructions on different datasets. Standard for modern distributed and multi-core systems.
 - **Shared Memory MIMD (Tightly Coupled):** Connected to a single global memory. Prone to memory contention, less tolerant to failures, and harder to extend.
 - **Distributed Memory MIMD (Loosely Coupled):** Each PE has local memory; communicates via an interconnection network. More scalable, fault-tolerant.

2. SPMD vs MPMD

- **SPMD:** Single Program Multiple Data (proposed by Federica Dardes). Every node runs the exact same code, but works on different chunks of data.
- **MPMD:** Multiple Program Multiple Data. Manager-worker model where nodes run completely different programs (heterogeneous tasks).

3. Types of Parallelism

- **Hardware-Level:** Pipelining, Superscalar, VLIW, SIMD, Multi-core, Clusters.
- **Software-Level:** Instruction-Level (ILP), Data-Level (DLP), Task-Level (TLP).

4. Needs & Limitations

- **Why Needed:** Serial computation wastes hardware potential; real-world data requires faster crunching.
- **Applications:** Databases, AI/ML, Real-time systems, VR/AR.
- **Limitations:** Synchronization overhead, algorithm design complexity, portability issues.

Lecture 3: Concurrency & Processors

1. Concurrency vs Parallelism

- **Concurrency:** OS handles multiple tasks by quickly context-switching between them (illusion of simultaneous execution). Focuses on structure and scheduling.
- **Parallelism:** Tasks run physically at the exact same time on multiple processing cores.
- **Key Difference:** Concurrency addresses multiple tasks at once but not necessarily simultaneously; parallelism executes tasks at the same time using different processors/cores.

2. Motivation for Concurrent Systems

- Physical resource sharing (limited hardware for multiple users).
- Logical resource sharing (shared files/data).
- Speedup through overlapping execution and modular design.

3. Independent vs Cooperating Processes

- **Independent:** State is not shared; output is deterministic/predictable.
- **Cooperating:** State is shared; output is non-deterministic (depends on execution order).

4. Process Operations

- **Creation:** Parent creates child (execute concurrently or wait; share resources or isolate).
- **Termination:** Parent can terminate child if resources exceeded or task done. Cascading rule: if a parent is terminated by the OS, its children must also be terminated.

5. Concurrency Issues, Advantages & Drawbacks

- **Issues:** Inconsistency, loss of data, deadlocks, race conditions, starvation, blocking.
- **Advantages:** Running multiple applications, better resource utilization, vastly improved response times.
- **Drawbacks:** Difficult to coordinate, heavy overhead on OS, hard to track bugs. Limits parallelism and introduces overhead/deadlock risks with critical sections.

6. Microprocessor Types

- **Scalar:** One data element at a time.
- **Vector/Array:** Multiple data elements.

- **Superscalar:** Multiple pipelines executing more than one instruction per clock cycle for enhanced performance.

7. RISC vs CISC

- **RISC:** Simple instructions (1 cycle), fixed length, heavily pipelined, more general-purpose registers. Hardwired control. Needs more lines of code.
 - **Advantages:** Simpler instruction decoding, faster execution, lower power consumption.
 - **Disadvantages:** More instructions required for complex tasks, increased memory usage.
- **CISC:** Complex instructions (multi-cycle), variable length, fewer registers. Uses both hardwired and microprogrammed control. Less code needed.
 - **Advantages:** Reduced code size, memory-efficient, widely used with large software base.
 - **Disadvantages:** Slower execution due to complex instruction decoding, higher power consumption.
- **CPU Performance:** RISC reduces CPI but increases instructions per program; CISC reduces instructions per program but can increase CPI.